CS 112

Introduction to Data Structures

WEEK 09

Stacks, Queues, and Hash Tables

Eric Araújo

Calvin University · Fall 2026

This Week

1. Abstract Data Types
2. The Stack ADT: LIFO
3. The Queue ADT: FIFO
4. Why the naive array queue fails
5. Circular arrays **KEY**
6. Hash tables: one hash, one bucket
7. Collisions, and what makes a good hash

Abstract Data Type

An ADT is a **promise**: which operations you may call, what values it holds, what each one costs. Never how the data is stored.

The ADT: what the user sees

Stack · last in, first out

`push(x)` `pop()`

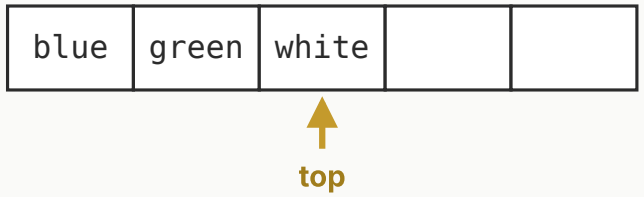
`peekTop()` `isEmpty()`

every button: O(1)

the user never looks past this line

Inside: a data structure

an array, top at the end



or a linked list, top at the front



The **Stack is an ADT**. An array or a linked list is a **data structure**: one way to keep the promise.

ADTs and How They Are Built

ADT	Operations	Promise	Built with
List	`get(i)`, `insert`, `remove`	items in positions 0 to n−1	dynamic array (your `Vec`, week 4) or linked list (your `List`, week 6)
Stack	`push`, `pop`, `peekTop`	last in, first out	array or linked list: **today**
Queue	`add`, `remove`, `peekFirst`, `peekLast`	first in, first out	circular array or linked list: **Wednesday**
Map	`put(key, value)`, `get(key)`	one value per key	hash table (**Friday**) or search tree (week 11)
Set	`add`, `contains`, `remove`	no duplicates	hash table or search tree (week 11)

Python's `list`, `dict` and `set` from CS108 are three of these, built for you. One ADT, several data structures: choosing between them is what this course is about.

PART 1 OF 3

Stacks

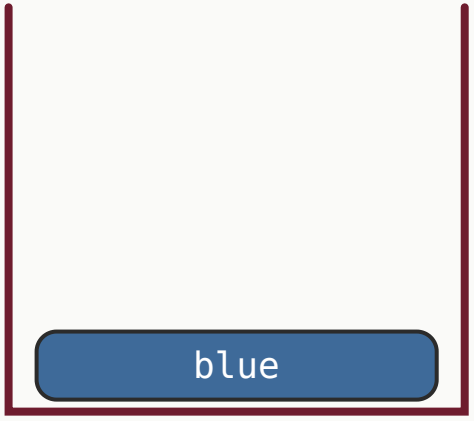
last in, first out



The Stack: Last In, First Out

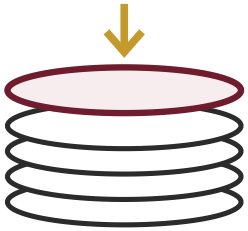
```
s.push("blue");
s.push("green");
s.push("white");
s.peekTop();  // "white", still there
s.pop();      // "white"
s.pop();      // "green"
```

Last in, first out: **white** went on last and comes off first. `push`, `pop` and `peekTop` all work at the **top**, and only there.



only the top is reachable

Stacks Are Everywhere



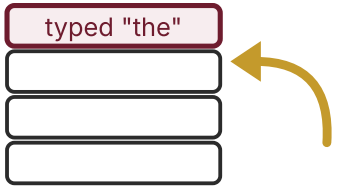
Plates

you take the one put back last



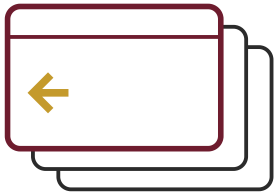
A tube of chips

only the last one in is within reach



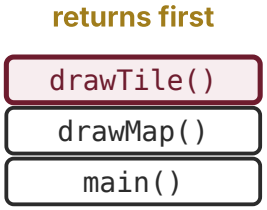
Undo, Ctrl-Z

the last change is the first undone



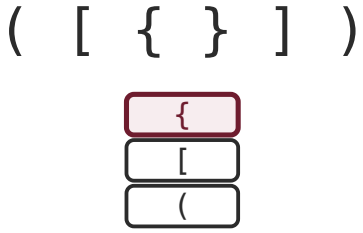
Back button

the page you just left is on top



Function calls

the last function called returns first



Matching brackets

push an opener, pop it at its closer

TALK TO YOUR NEIGHBOR · TTYN

What does this print?

```
s.push(7);
s.push(8);
s.push(9);
while (!s.isEmpty()) {
    cout << s.pop() << " ";
}
```

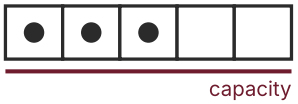
A. 7 8 9

B. 9 8 7

C. 9 7 8

D. 7 9 8

What Goes Inside a Stack?



An array

fixed size: it can be full



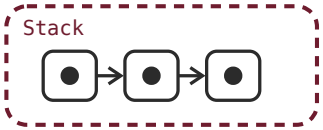
A dynamic array

grows when full, like your `Vec`



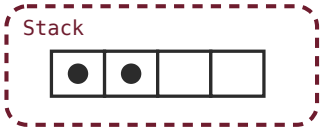
A linked list

nodes and pointers, like your `List`



An internal `List`

reuse your `List` class inside



An internal `Vec`

reuse your `Vec` class inside

Where is the top? What does it cost?

TALK TO YOUR NEIGHBOR · TTYN

A stack in an array: where should the top be?

A. Always at index 0

B. Always at index `mySize - 1`

C. It does not matter, both are equally fast

The Array Stack

```
s.push("a");
s.push("b");
s.push("c");
s.pop();      // "c" stays
s.push("d"); // over "c"
```

mySize = 3



myCapacity = 6

Nothing shifts: O(1). Full when `mySize == myCapacity`.

TALK TO YOUR NEIGHBOR · TTYN

A stack in a singly linked list: where should the top be?

A. At the front, `myFirst`

B. At the back, `myLast`

C. It does not matter, both are equally fast

A Stack Built on Your `List`

```
template <class Item>
class Stack {
public:
    void push(const Item& it) { myList.prepend(it); }
    Item pop()                { return myList.removeFirst(); }
    Item peekTop() const      { return myList.getFirst(); }
    bool isEmpty() const      { return myList.getSize() == 0; }
private:
    List<Item> myList;   // used only at its front
};
```

every method: **one line** | already written and tested | top = front: O(1) | cost: a new per item

Five Ways to Build a Stack

Inside	Speed	Memory	Easy to write	Watch out for
Array	●●●	●●●	●●○	a maximum size
Dynamic array	●●○	●●○	●●○	push is O(1)* on average
Linked list	●○○	○○○	●○○	pointers are tricky
Internal `Vec`	●○○	●●○	●●●	O(1)* push, from `Vec`
Internal `List`	●○○	○○○	●●●	memory per item

a10 builds the first row.

PART 2 OF 3

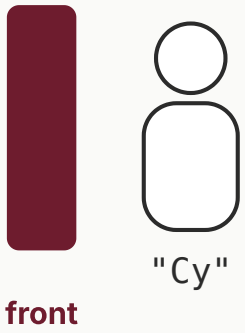
Queues

first in, first out

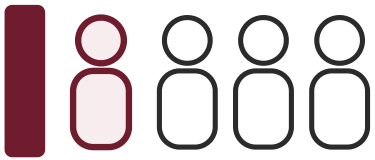


The Queue: First In, First Out

```
q.add("Ana");
q.add("Ben");
q.add("Cy");
q.peekFirst();  // "Ana"
q.peekLast();   // "Cy"
q.remove();     // "Ana"
q.remove();     // "Ben"
```

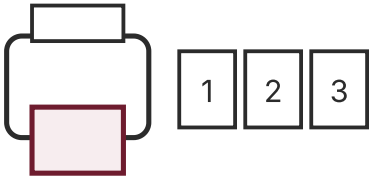


Queues Are Everywhere



A line at a counter

first to arrive, first served



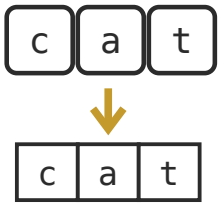
Print jobs

pages print in the order they were sent



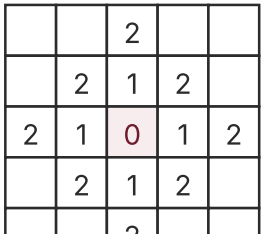
Up next

the song added first plays first



Keystrokes

typed c, a, t: read as c, a, t



Searching a maze

explore the nearest cells first



A snake game

the head joins, the tail leaves

Whenever **first come, first served** is the rule, a queue is the structure.

TALK TO YOUR NEIGHBOR · TTYN

What does this print?

```
Stack<int> s(5);
Queue<int> q(5);

s.push(10);
q.add(s.peekTop());
q.add(11);
cout << q.peekFirst() << " ";
s.push(q.peekFirst());
s.push(13);
cout << s.peekTop() << " ";
q.add(12);
s.push(q.peekLast());
cout << s.peekTop() << " " << q.peekFirst();
```

A. 10 13 12 10

B. 10 10 13 12

C. 10 11 13 12

D. 10 11 12 13

TALK TO YOUR NEIGHBOR · TTYN

A queue in a linked list with `myFirst` and `myLast`: where should the front be?

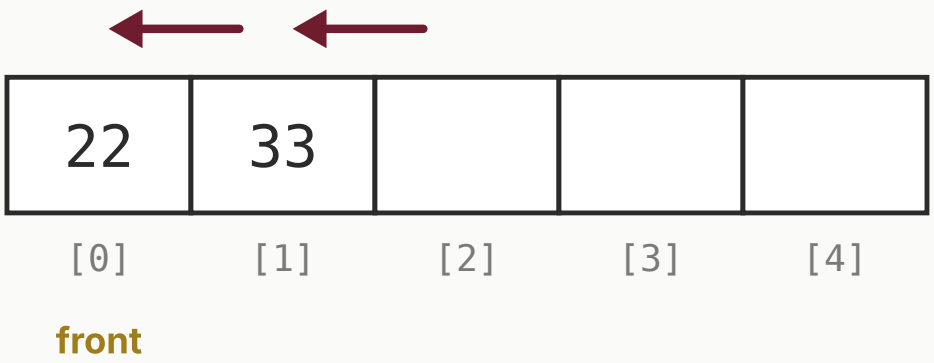
A. At `myFirst`

B. At `myLast`

C. It does not matter, both are equally fast

The Obvious Array Queue

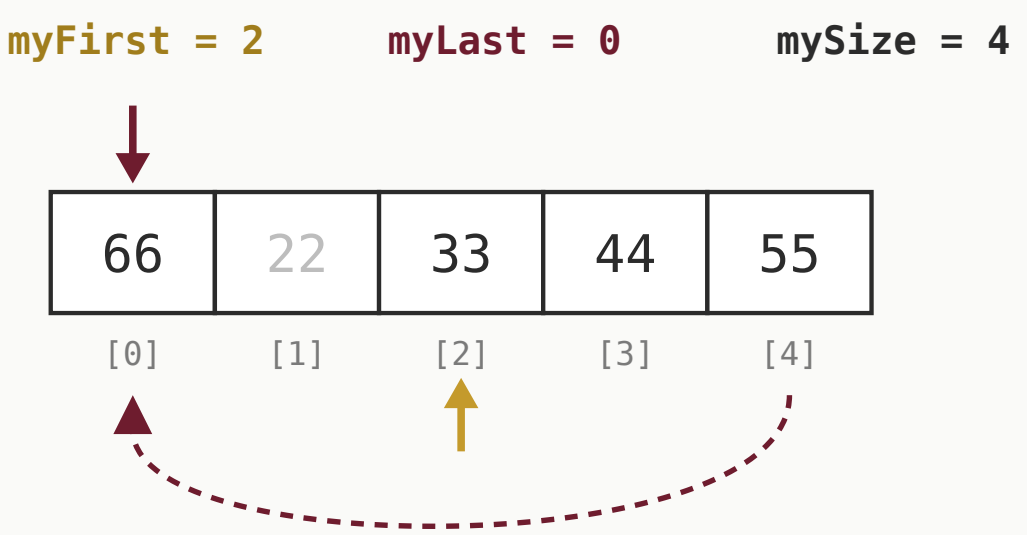
```
q.add(11);
q.add(22);
q.add(33);
q.remove();  // 11
```



Every remove moves every item: **O(n)**.

Move the Front, Not the Items

```
q.add(11);
q.add(22);
q.add(33);
q.remove();   // 11
q.remove();   // 22
q.add(44);
q.add(55);
q.add(66);   // where?
```



One Step Forward

```
++i;
if (i == myCapacity) {
    i = 0;
}
```

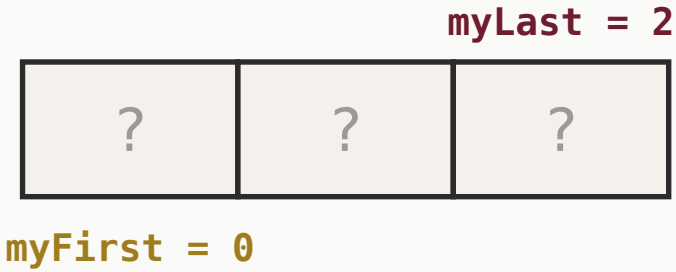
In one line, with no `if`?

```
i = (i + 1) % myCapacity;
```

0 → 1 → 2 → 3 → 4 → **0** → 1 → …

TALK TO YOUR NEIGHBOR · TTYN

No `mySize`. Capacity 3, `myFirst` is 0, `myLast` is 2. Empty or full?



A. Empty

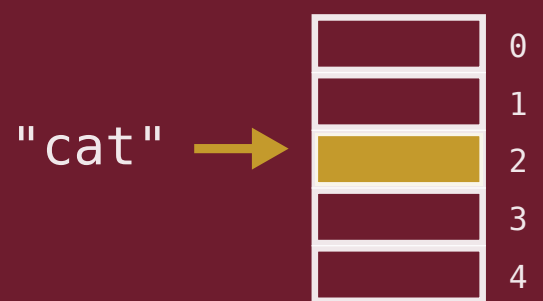
B. Full

C. Can't tell

PART 3 OF 3

Hash Tables

find it in one step

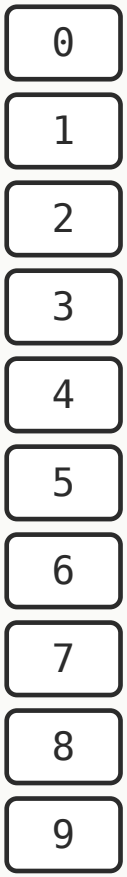


Can We Beat O(lg n)?

Structure	Insert	Remove	Search
vector	`push_back` O(1)*	`pop_back` O(1)	O(n)
linked list	`push_front` O(1)	`pop_front` O(1)	O(n)
stack, queue	O(1)	O(1)	none
BST, `set`, `map` search trees: week 11	O(lg n)	O(lg n)	O(lg n)
???	**O(1)**	**O(1)**	**O(1)**

A Vector of Buckets

```
class HashTable {
public:
    void insert(const string& word);
    bool contains(const string& word) const;
    ...
private:
    vector< list<string> > myTable;
};
```

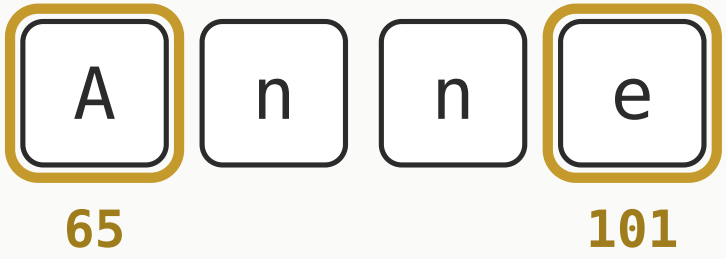


each list is a **bucket**

A Simple Hash Function

```
unsigned HashTable::hash(const string& word) const {
    return (word[0] + word[word.length() - 1]) % TABLE_SIZE;
}
```

hash("Anne"), with TABLE_SIZE 10:



65 + 101 = 166

166 % 10 = 6

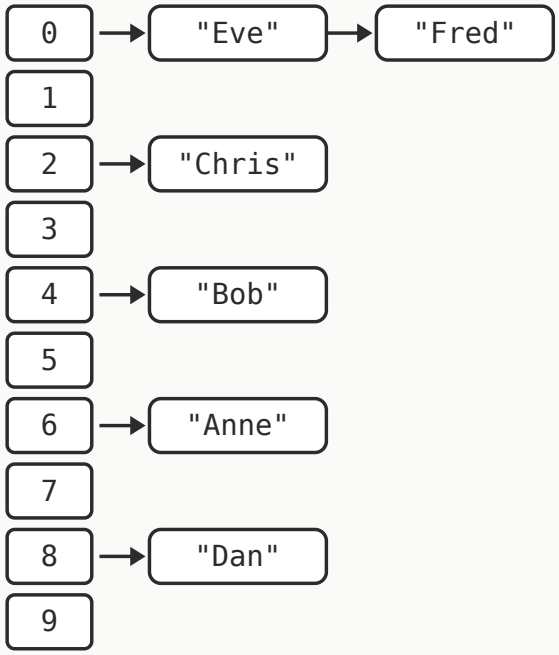
bucket 6

How long does it take? O(1): two characters and a `%`, however many words are in the table.

`insert`, Step by Step

```
t.insert("Anne");
t.insert("Bob");
t.insert("Chris");
t.insert("Dan");
t.insert("Eve");
t.insert("Fred");
```

TABLE_SIZE is 10 here



`'F' + 'd' = 70 + 100 = 170`

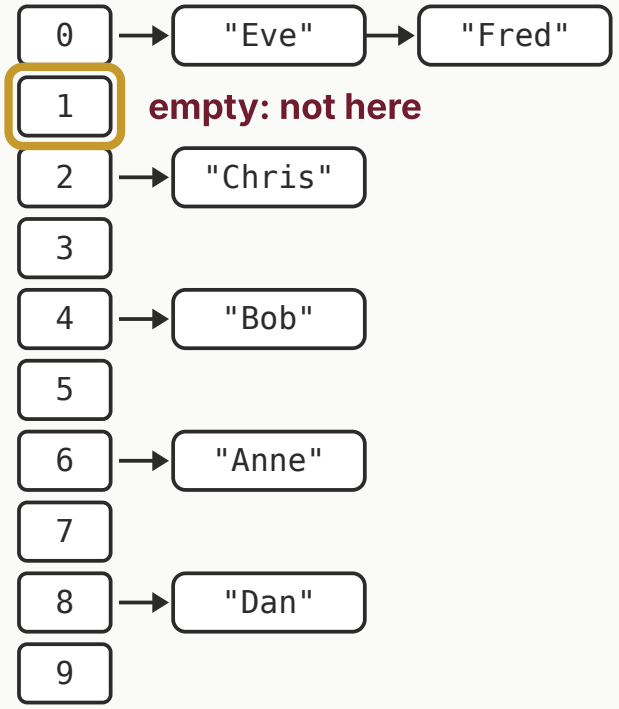
`170 % 10 = 0`

a collision:
same bucket as "Eve"

contains

```
t.contains("Bob");   // true
t.contains("Zoe");   // false
```

One hash, then **one** bucket. Never the rest of the table.



How Is `contains` O(1)?

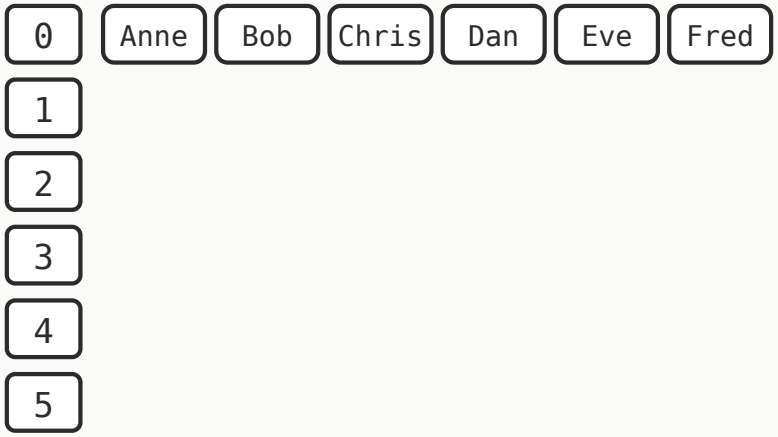
It walks a bucket's list, and walking a list is O(n).

Yes. But what is n?

The length of **one bucket**, not the size of the table.



spread: walk 1 name



one bucket: walk all 6

Enough buckets and a hash that spreads: n is about 1, so O(1).
Everything in one bucket: O(n).

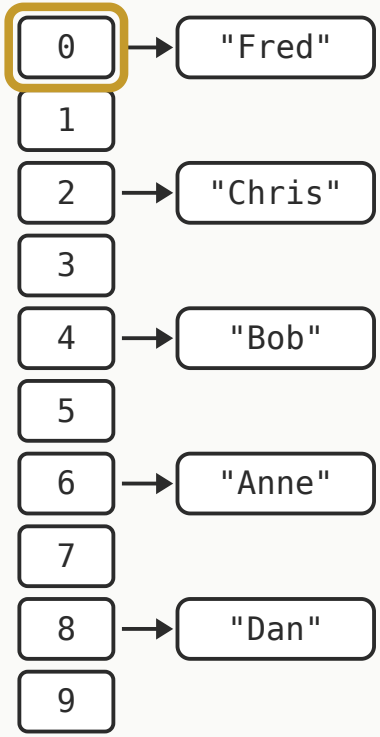
Your Turn: `remove("Eve")`

What are the steps?

1. `hash("Eve")`: bucket 0
2. take `"Eve"` out of that bucket's list

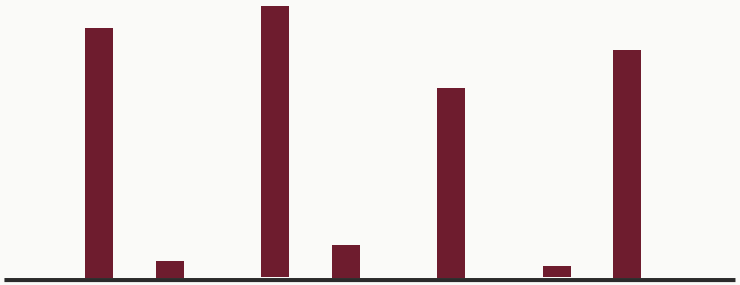
What does it cost?

Hash: O(1). Walk one bucket: about 1 long. So O(1), like `contains`.



A Good Hash Function

1. **Fast**: O(1), it never looks at the table
2. **Spreads** words evenly across the buckets
3. **Deterministic**: the same word, the same bucket, every time



clumped: a few long buckets

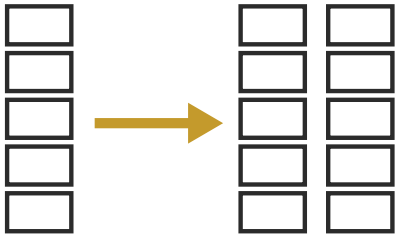


spread: every bucket short

First + last character: fast and deterministic. Does it spread? You measure that in a11.

Too Many Collisions?

Load factor = words / buckets. Rule of thumb: 2 buckets per word.



5 buckets → 10, rehash all

1. More buckets



taken? try the next one

2. Probe



a second hash picks again

3. Hash again

Are 2 and 3 still O(1)?

On average, yes: with the load factor low, a free bucket is a step or two away. As the table fills, the search gets long, up to O(n). So real tables add buckets (1) before that happens.

The STL Has Them

```
#include <unordered_set>
#include <unordered_map>

unordered_set<string> words;
words.insert("gargoyle");
words.count("gargoyle");     // 1

unordered_map<string, int> age;
age["Anne"] = 19;
```

	inside	order	search
`set` `map`	tree	sorted	O(lg n)
`unordered_set` `unordered_map`	hash table	none	**O(1)**

Can We Beat O(lg n)? Yes.

Structure	Insert	Remove	Search
vector	`push_back` O(1)*	`pop_back` O(1)	O(n)
linked list	`push_front` O(1)	`pop_front` O(1)	O(n)
stack, queue	O(1)	O(1)	none
BST, `set`, `map` search trees: week 11	O(lg n)	O(lg n)	O(lg n)
hash table, `unordered_set`	**O(1)**	**O(1)**	**O(1)**

O(1) on average. The price: no order, and a hash function to get right.

Week 09 Recap

- An **ADT** fixes the behavior and leaves the storage open
- Stack = LIFO; Queue = FIFO; a hash table keeps no order at all
- Put the action where the structure is fast: end of an array, front of a list
- A naive array queue makes `remove()` O(n); a **circular array** fixes it
- `(i + 1) % myCapacity` is the whole trick
- `myFirst` starts at 0, `myLast` at capacity - 1; `mySize` tells full from empty
- A **hash table** computes the index: one hash, one bucket, O(1) on average

Next: a10 · a11 →

This deck stands on earlier CS112 materials by
Joel Adams and **Victor Norman** · adapted and extended by **Eric Araújo**